



DESIGN

DOC NO. RL-370-DESIGN-A REV. A RIPOSTE LABORATORIES

How glitchlock works, why each choice was made, and the measurements behind them. Everything in the "Evidence" section was run against FFglitch 0.10.2 on x86-64 Linux and is reproducible with glitchlock verify.

CONTENTS

▶ 1. The problem	3
▶ 2. Plaintext is the carrier, not your source file	3
▶ 3. Slots and domains	4
3.1 Motion vectors	4
3.2 Quantiser scale	5
3.3 What was rejected	5
▶ 4. The transform	5
4.1 The plan is data-independent	6
4.2 Keystream	6
▶ 5. Verification, not optimism	7
▶ 6. Evidence	7
Identity apply is byte-exact	7
Motion vectors wrap, they do not clamp	8
Full-domain randomisation survives exactly	8
DCT coefficients do not survive	8
End to end	9
▶ 7. Known limits	10

SEC.01

1. THE PROBLEM

Datamoshing corrupts a compressed bitstream to produce visual artefacts. It is destructive by nature: the interesting glitches come precisely from feeding the decoder values the encoder never intended, and there is no general way back.

We want the same visual result from an operation that is a **bijection**, with a small sidecar recording the parameters.

Three things have to be true:

1. Every value we write must be one the codec can store and return **exactly**.
2. The transform must be invertible given a key.
3. The **set of editable positions** must be identical before and after, or the inverse has nowhere to put the values back.

Point 3 is the one that quietly kills most candidate features.

SEC.02

2. PLAINTEXT IS THE CARRIER, NOT YOUR SOURCE FILE

`glitchlock prepare` transcodes arbitrary input into an MPEG-2 or MPEG-4 elementary stream. That step is an ordinary lossy transcode and it drops audio.

Everything downstream is bit-exact, and the carrier is what the manifest's `carrier_sha256` commits to. This boundary is deliberate and stated plainly rather than blurred: `glitchlock` does not claim to give you back your original H.264 file, because it cannot.

SEC.03

3. SLOTS AND DOMAINS

A **slot** is one mutable scalar integer in an FFedit feature payload. A **domain** is the finite set of values that slot may hold, written `(lo, n)` meaning `lo .. lo+n-1`.

Slots are enumerated in a canonical order (lists by index, dicts by sorted key, null cells skipped because a null means "this block is not coded" and writing there would change the bitstream structure). Both directions of the transform walk the same order, so slot `i` on the way out is slot `i` on the way back.

3.1 MOTION VECTORS

In both MPEG-2 and MPEG-4 the motion vector is reconstructed as `predictor + delta` and then sign-extended to a fixed width tied to the frame's `f_code`. FFmpeg does this with `sign_extend(val, W + f_code)`. The consequence is the whole basis of this tool: **out-of-range writes wrap, they do not clamp**. A wrapping field is a cyclic group, and modular arithmetic on it is exactly invertible.

`W` differs by codec, and this is the single most dangerous detail in the project:

CODEC	W	N AT F_CODE=1	DOMAIN
MPEG-1/2 video	4	32	<code>[-16, 15]</code>
MPEG-4 / H.263 family	5	64	<code>[-32, 31]</code>

An MPEG-4 stream legitimately contains the vector `17` at `f_code=1`. Assume the MPEG-2 width and that becomes `-15`, silently, permanently. This was not a theoretical concern – it was a real bug caught by the integration suite during development, and it is now covered by a named regression test.

`f_code` is per frame. On MPEG-2 it is also per axis, so `x` and `y` in the same `[x, y]` pair can have different domains. The `f_code` values

themselves are never modified.

3.2 QUANTISER SCALE

MPEG-2 `q_scale_code` is a fixed-width 5-bit field with legal values 1..31, so the domain is (1, 31). Because the field width is fixed, changing the value cannot shift anything downstream.

3.3 WHAT WAS REJECTED

DCT coefficients (`q_dc`, `q_dct`, and their delta variants) are variable-length coded. Changing a value changes its bit length, which moves every subsequent bit in the slice. Measured, not assumed – see Evidence below.

`mv_delta` is rejected for a subtler reason: the delta's legal range depends on the predictor, which depends on neighbouring vectors, so there is no fixed per-slot domain to be modular over.

`mb` (macroblock types) is rejected because the type determines how the **following** data is parsed – change it and the payload you were going to edit no longer means the same thing.

SEC.04

4. THE TRANSFORM

Two keyed layers per frame, both bijections on the slot domains.

Substitution. Each selected slot gets its own keystream offset:

```
lock:    v' = lo + ((v - lo) + k) mod n
unlock:  v  = lo + ((v' - lo) - k) mod n
```

Permutation. Selected slots are bucketed by **identical domain** and shuffled within their bucket by a keyed Fisher-Yates. The bucketing is load-bearing: moving a value from an `f_code=5` frame into an `f_code=1` slot would push it outside the smaller domain, and the codec would wrap it. Values only move between slots that can represent them.

Order is substitute → permute when locking, inverse-permute → inverse-substitute when unlocking.

4.1 THE PLAN IS DATA-INDEPENDENT

All keystream is consumed to build a `FramePlan` – selection mask, offsets, bucket permutations – **before** any value is read. The plan therefore depends only on (`key`, `nonce`, `mode`, `intensity`, `slot geometry`).

This matters because unlock has to rebuild the identical plan while holding only the ciphertext. Slot geometry is invariant by construction (we never change `f_code`, never fill a null, never alter a macroblock type), so the ciphertext yields the same geometry and hence the same plan.

4.2 KEYSTREAM

HMAC-SHA256 in counter mode, domain-separated per `feature | stream | frame | part`. Integers are drawn by rejection sampling, so there is no modulo bias – a biased sampler would still be reversible but would leak structure.

`intensity < 1` selects a key-derived subset of slots. Since the selection comes from the keystream and not from the data, unlock reproduces it exactly. Partial intensity weakens the **visual** scrambling, never the reversibility.

SEC.05

5. VERIFICATION, NOT OPTIMISM

Two independent checks, because a scheme that is reversible in theory and broken in practice is worse than one that admits it.

Per-layer diff. After applying a layer, glitchlock re-exports the feature from its own output and compares it against what it intended to write. Any slot that disagrees is recorded in the manifest's `repairs` map as the **original plaintext value**. Unlock applies those after inverting, so recovery is exact even if a codec edge case bites. If the slot **count** differs — meaning the geometry moved — that is unrecoverable and the lock aborts.

On `mv` and `qscale`, `repairs` is empty. The test suite asserts that rather than trusting it.

End-to-end self-test. `lock` then unlocks its own output into a temp directory and compares SHA-256 against the input carrier. On failure it raises and **writes no manifest**. You cannot end up holding a locked file plus a manifest that does not actually unlock it.

SEC.06

6. EVIDENCE

Measurements from FFglitch 0.10.2, MPEG-2 320×240 and 640×480 test clips.

IDENTITY APPLY IS BYTE-EXACT

Exporting a feature and applying it back unchanged reproduces the input file byte for byte, on both codecs. This is the floor the whole design stands on: if it were false, no amount of correct arithmetic would give back the original bytes.

```
carrier_mpeg2.mpg 3d1528bf... -> null_apply.mpg 3d1528bf... identical
carrier_mpeg4.mp4 7f832b2f... -> mp4_null.mp4 7f832b2f... identical
```

MOTION VECTORS WRAP, THEY DO NOT CLAMP

Writing probe values around the boundary at `f_code=1` and reading them back:

WROTE	MPEG-2 GOT	MOD-32 PREDICTS	MPEG-4 GOT	MOD-64 PREDICTS
-33	-1	-1	31	31
-17	15	15	-17	-17
-16	-16	-16	-16	-16
15	15	15	15	15
16	-16	-16	16	16
31	-1	-1	31	31
32	0	0	-32	-32
47	15	15	-17	-17

Every probe matches the modular prediction for its codec's width, and only for its codec's width. MPEG-2 fits size 32 and not 64; MPEG-4 fits 64 and not 32.

FULL-DOMAIN RANDOMISATION SURVIVES EXACTLY

CARRIER	SLOTS RANDOMISED ACROSS FULL DOMAIN	MISMATCHES ON RE-EXPORT
MPEG-2, f_code 1	28,800	0
MPEG-2, f_codes 1-5 mixed	14,400	0
MPEG-4, f_code 1	57,600	0
MPEG-2 qscale	1,500	0

DCT COEFFICIENTS DO NOT SURVIVE

The same experiment on `q_dc` desynchronises the bitstream. The decoder emits `ac-tex damaged`, `invalid cbp`, `slice mismatch` and `mb incr damaged`, and – fatally – the re-exported payload has a **different number of slots** than the one written:

```
STRUCTURE CHANGED 254 -> 223
STRUCTURE CHANGED 279 -> 228
```

STRUCTURE CHANGED 230 -> 212
...94 of 100 frames affected

There is no manifest that repairs a structural change, so the feature is refused rather than offered with a warning.

END TO END

10 s, 640×480, MPEG-2 carrier, mv + qscale, full mode:

slots scrambled	583,500
lock (including self-test)	7.9 s
unlock	3.4 s
carrier	3,263,287 bytes
locked	3,790,607 bytes (+16%)
restored	3,263,287 bytes, SHA-256 identical
manifest	1,085 bytes
repairs recorded	0

The locked file grows because scrambled vectors are less predictable and cost more bits to code. This is expected and harmless.

SEC.07

7. KNOWN LIMITS

- ▶ `prepare` is lossy and drops audio; the carrier is the plaintext.
- ▶ Only MPEG-2 and MPEG-4 part 2 are round-trip tested. Other codecs in the domain table share MPEG-4's geometry but are untested here – `verify` will tell you before you trust one.
- ▶ The locked file is larger than the carrier.
- ▶ Reversibility is tied to FFglitch's behaviour. A future release that changes MV export semantics would be caught by the self-test, which is exactly why the self-test runs by default.

SOURCE DESIGN.tex SHA 9ce7238cf689 BUILT 2026-09-07